Machine Learning Operations (MLOps) AIMLCZG523

Heart Disease Risk Prediction: End-to-End MLOps Pipeline

Assignment 01 - Final Report

Student	[Your Name]
Student ID	2025ae05981
Code repository	https://github.com/2025ae05981-debug/heart-disease-mlops
Demo video	[add your video link here]
Deployment	Local Kubernetes (Docker Desktop), http://localhost

1. Introduction and objectives

The goal of this assignment is to build a classifier that predicts the presence of heart disease from patient health data, and to deliver it as a reproducible, tested, containerised and monitored service. The work covers data preparation, model development, experiment tracking, packaging, CI/CD, containerisation, Kubernetes deployment, and monitoring.

The dataset is the UCI Heart Disease dataset (Cleveland subset, 303 patients, 13 features). The original target (0 to 4) is binarised: 0 means no disease and 1 means disease.

1.1 Architecture

Figure 1 shows how the pieces fit together. Training produces a single scikit-learn pipeline that is served by FastAPI inside a Docker image. Kubernetes runs two replicas behind a LoadBalancer. Prometheus scrapes the API metrics and Grafana displays them. GitHub Actions lints, tests and retrains on every push.

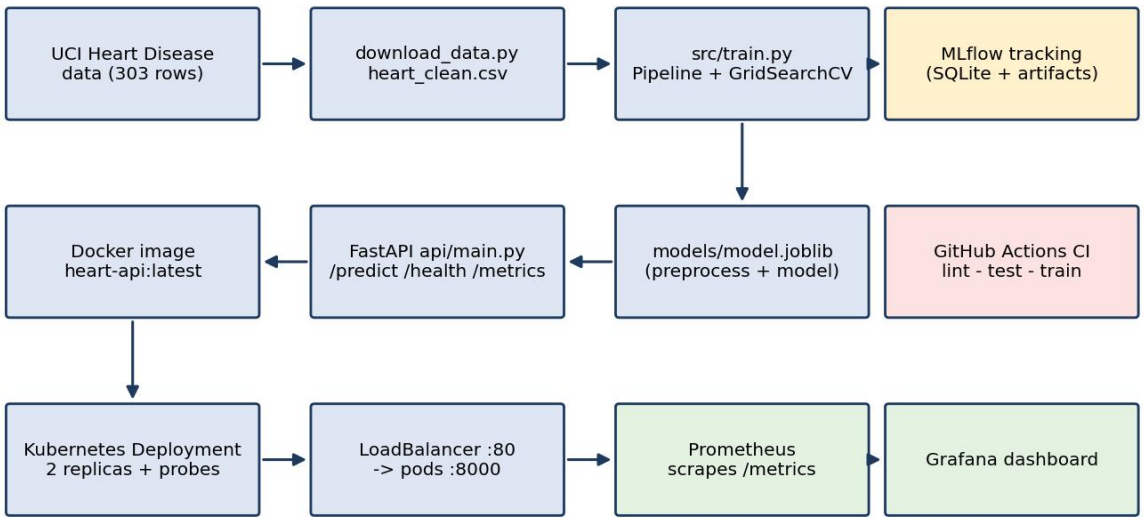


Figure 1. System architecture.

1.2 Technology summary

Area	Tools
Data and modelling	Python 3.13, pandas 3.0.6, numpy 2.5.3, scikit-learn 1.9.1, matplotlib, seaborn
Experiment tracking	MLflow 3.16.1 (SQLite backend)
Serving	FastAPI 0.142.2, uvicorn 0.54.0, prometheus-client 0.26.0
Quality	pytest 9.1.1, flake8 7.4.1, GitHub Actions
Deployment	Docker, Kubernetes (Docker Desktop, kubeadm v1.36.1)
Monitoring	Prometheus v2.55.1, Grafana 11.3.0

2. Setup and installation

Prerequisites: Windows with WSL2, Python 3.13, Git, and Docker Desktop with Kubernetes enabled. From a clean checkout:

```
git clone https://github.com/2025ae05981-debug/heart-disease-mlops
cd heart-disease-mlops
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python download_data.py
```

```
python -m src.train
python -m pytest tests -v
```

Reproducibility check. A second virtual environment was created from `requirements.txt` alone, and `python -m src.predict` loaded the saved model and returned a prediction. This confirmed that the project runs from a clean setup.

Dependencies are pinned in `requirements.txt` (training and development) and `requirements-api.txt` (a slim set for the container).

3. Data acquisition and exploratory analysis

`download_data.py` fetches the dataset with the `ucimlrepo` package (dataset id 45), keeps the raw file, binarises the target and writes `data/heart_clean.csv`. Missing values occur only in ca (4 rows) and thal (2 rows). The cleaned file has 303 rows: 164 without disease and 139 with disease, so the classes are nearly balanced and no resampling was needed.

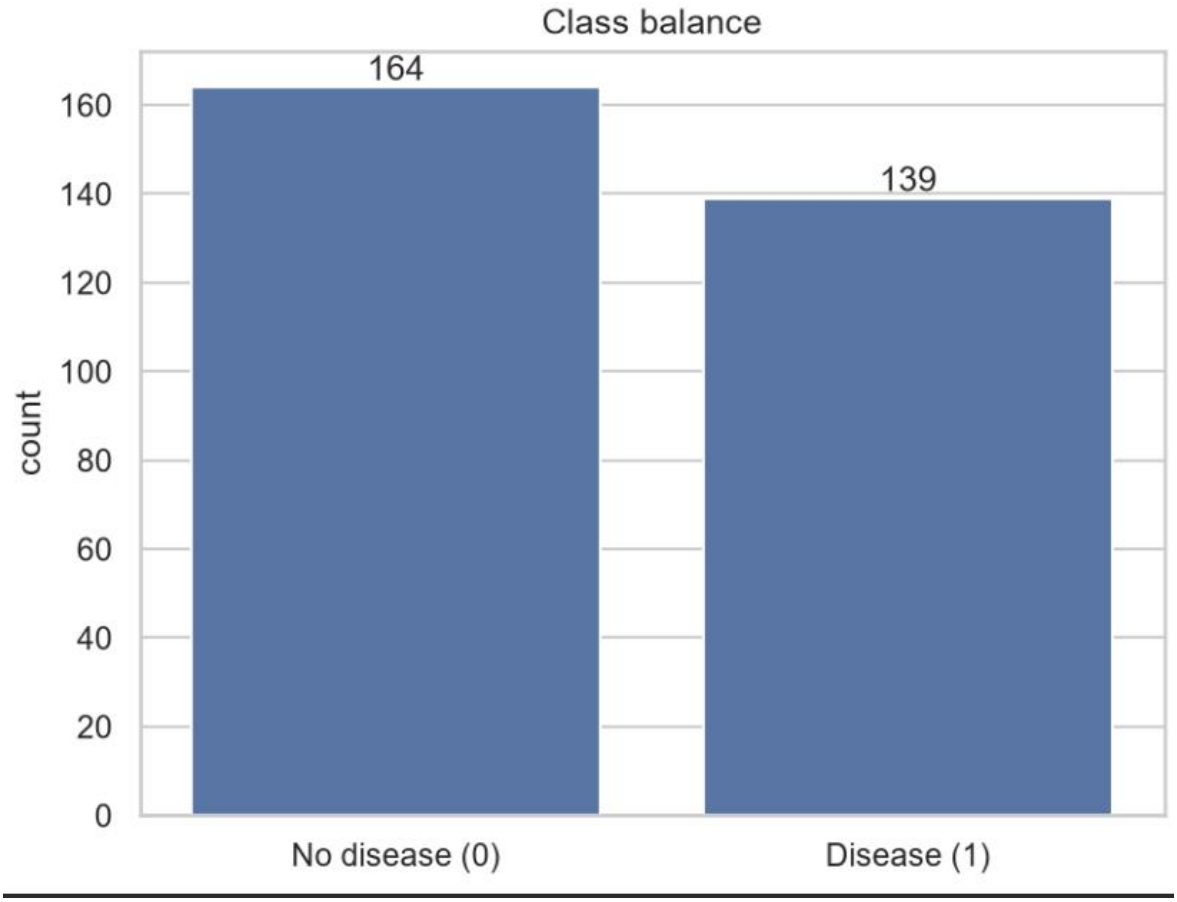


Figure 2. Class balance of the binary target (164 vs 139).

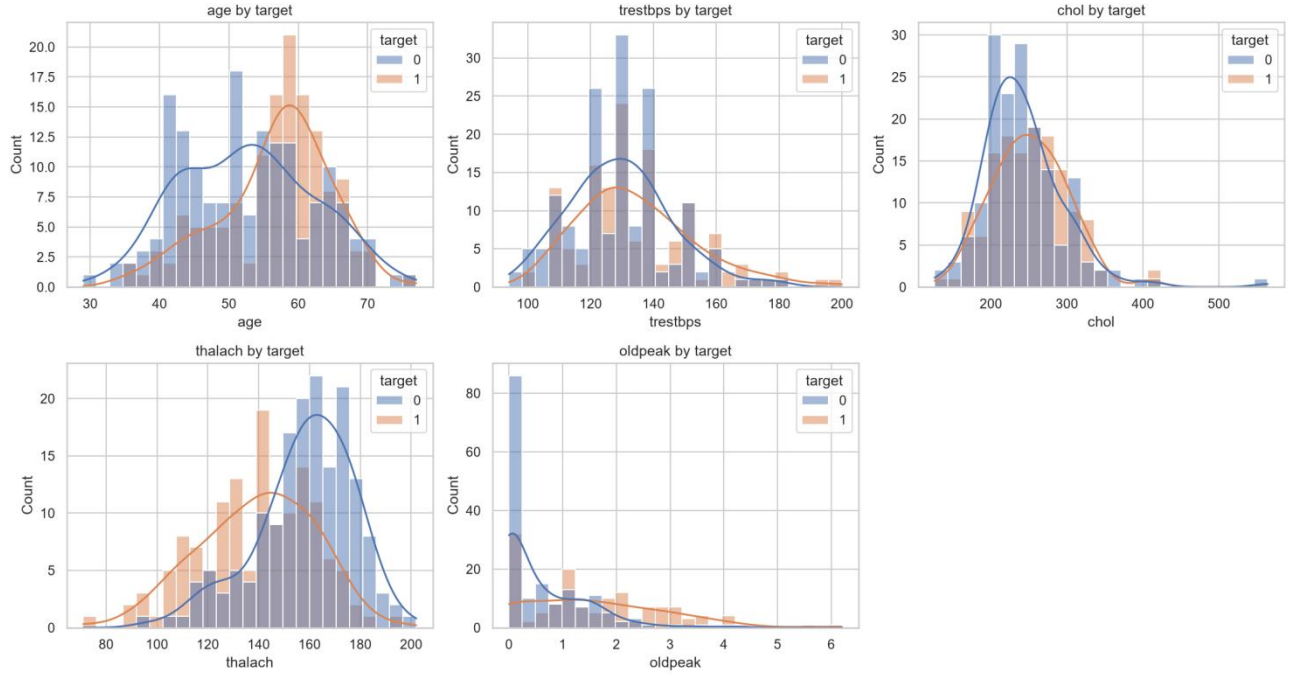


Figure 3. Distribution of the numeric features by target class.

3.1 Observations

- `thalach` (maximum heart rate) separates the classes best: patients with disease cluster around 120 to 145, healthy patients around 155 to 180.
- `oldpeak` is strongly right-skewed. Most healthy patients are near 0, while disease cases spread to 2 to 4 and above.
- `age` peaks around 55 to 60 for disease cases; healthy patients are spread over a wider and younger range.
- `chol` and `trestbps` overlap heavily between classes and have a few outliers (chol near 560, trestbps near 200).

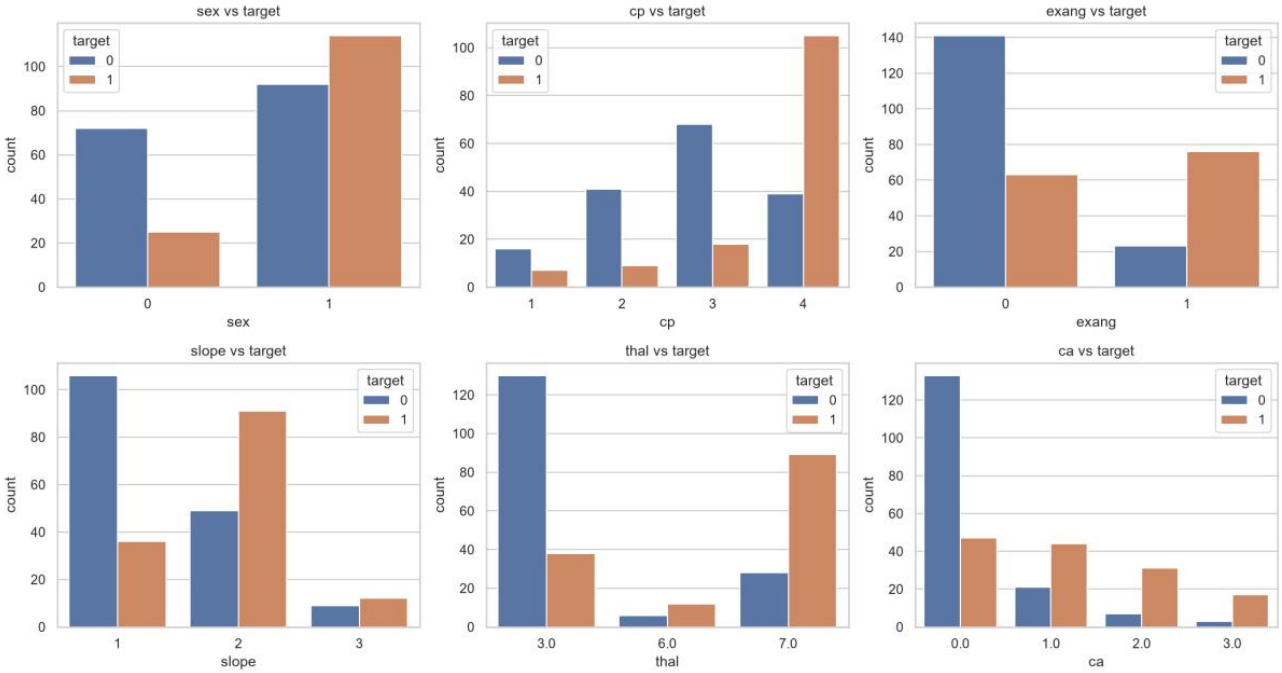


Figure 4. Categorical features against the target.

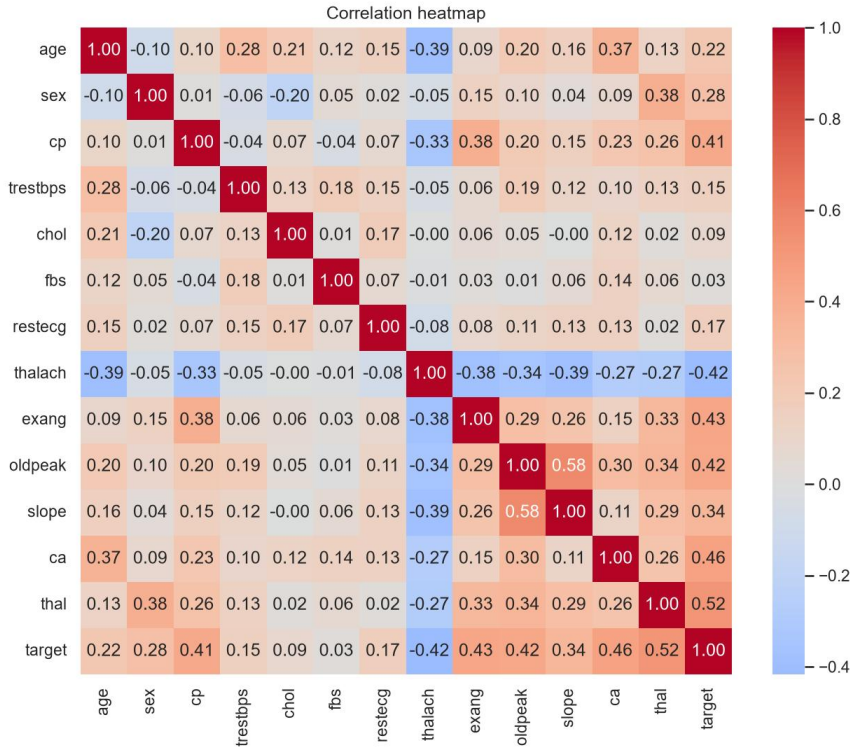


Figure 5. Correlation heatmap of all features and the target.

The strongest correlations with the target are `thal` (0.52), `ca` (0.46), `exang` (0.43), `oldpeak` (0.42), `cp` (0.41) and `thalach` (-0.42). `chol` (0.09) and `fbs` (0.03) are weak on their own. The highest correlation between two features is `oldpeak` and `slope` (0.58), which is moderate, so no feature was dropped. Pearson correlation is only a rough guide for coded categorical columns.

4. Feature engineering and model development

4.1 Preprocessing pipeline

All preprocessing lives in `src/preprocessing.py` as one scikit-learn `ColumnTransformer`:

- **Numeric** (age, trestbps, chol, thalach, oldpeak): median imputation, then standardisation.
- **Categorical** (sex, cp, fbs, restecg, exang, slope, ca, thal): most-frequent imputation, then one-hot encoding with `handle_unknown='ignore'`, so an unseen category at serving time does not crash the API.
- Imputation is inside the pipeline, so it is fitted only on training folds during cross-validation. This avoids data leakage.

4.2 Models, tuning and evaluation

Two classifiers were trained on a stratified 80/20 split (random state 42). Each was tuned with `GridSearchCV` using stratified 5-fold cross-validation and ROC-AUC as the selection metric. The test set was not touched during tuning.

Model	Search grid	Best parameters	CV ROC-AUC	Test ROC-AUC
Logistic Regression	C in {0.01, 0.1, 1, 10}	C = 1	0.902	0.966
Random Forest	n_estimators {100, 200}, max_depth {3, 5, None}, min_samples_split {2, 5}	n_estimators 100, max_depth None, min_samples_split 5	0.898	0.955

Accuracy, precision, recall and F1 were also computed (cross-validated and on the test set) and logged to MLflow. Logistic Regression was selected as the final model. The two models are close in cross-validation (0.902 against 0.898), and the simpler model is easier to explain and faster to serve. The test AUCs are higher than the CV values, but the test set has only 61 patients, so the cross-validated figures are the more reliable estimate.

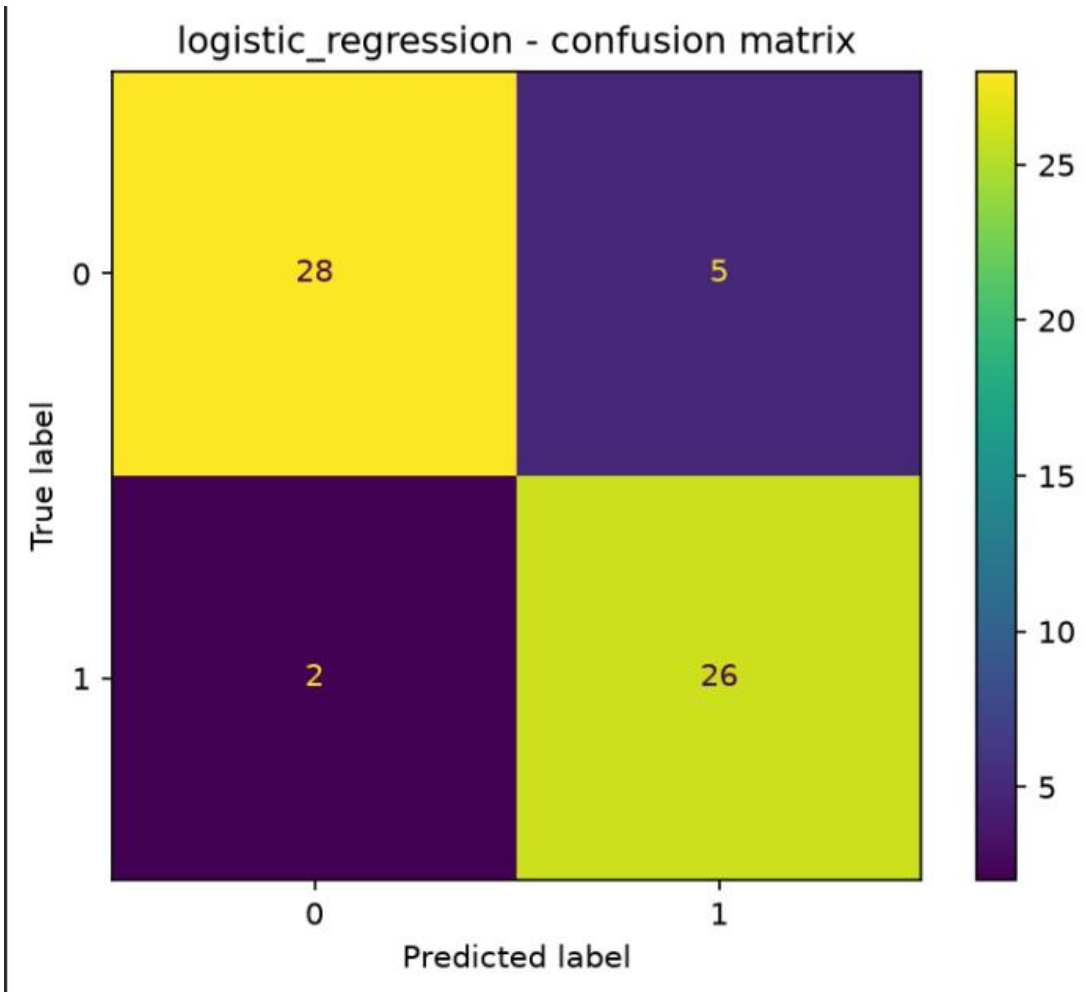


Figure 6. Confusion matrix of the selected Logistic Regression model on the test set.

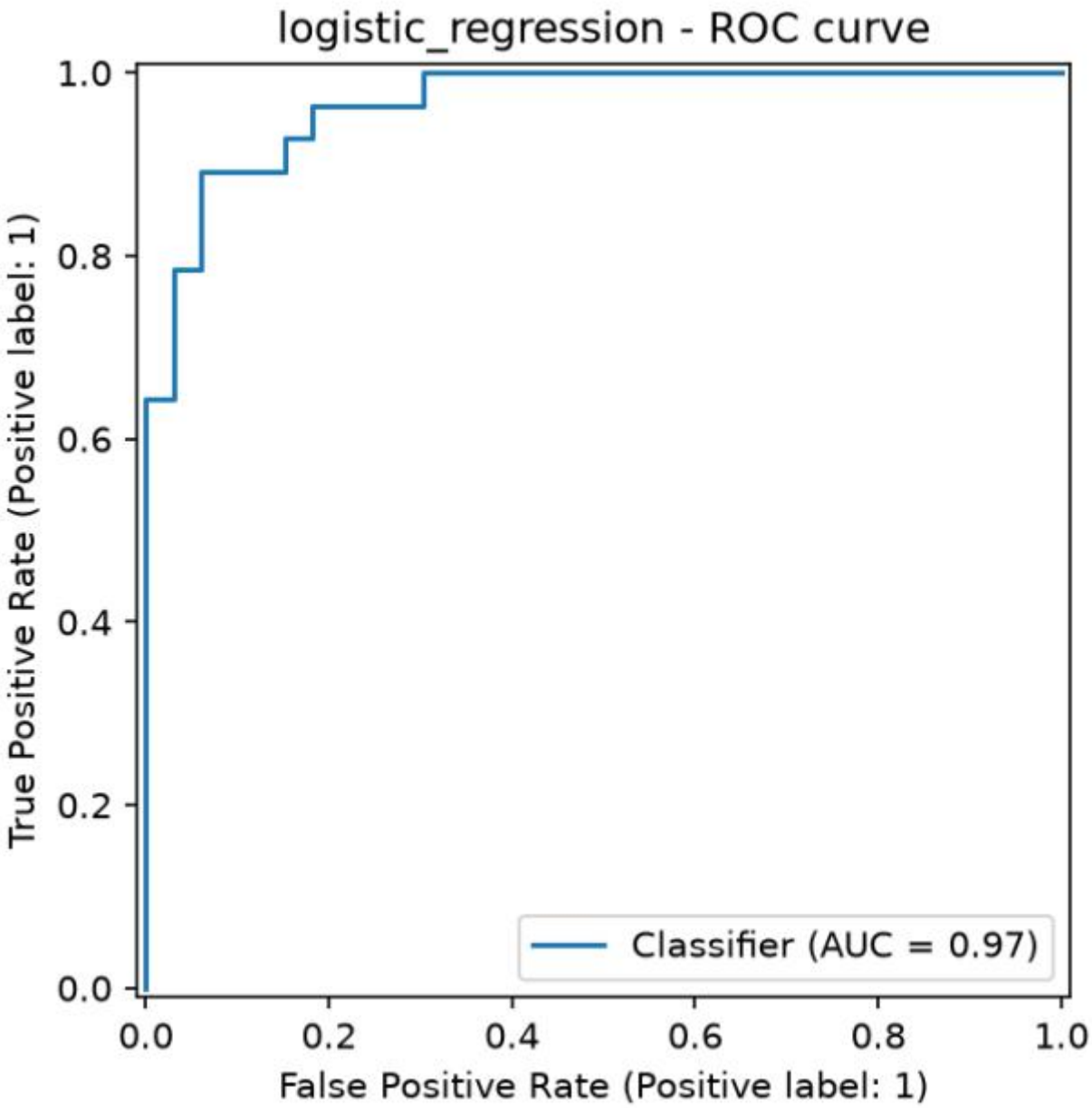


Figure 7. ROC curve of the selected model on the test set.

5. Experiment tracking

Every training run is tracked with MLflow. Each run logs the model type, the best hyperparameters, the mean and standard deviation of the cross-validated accuracy, precision, recall and ROC-AUC, the hold-out test metrics, the confusion matrix and ROC curve as artifacts, and the fitted pipeline as an MLflow model.

The tracking backend is a local SQLite database (`mlflow.db`). The default file store was rejected by MLflow 3.16, which places it in maintenance mode. The database and artifacts are excluded from Git, so the screenshots below are the record of the runs.

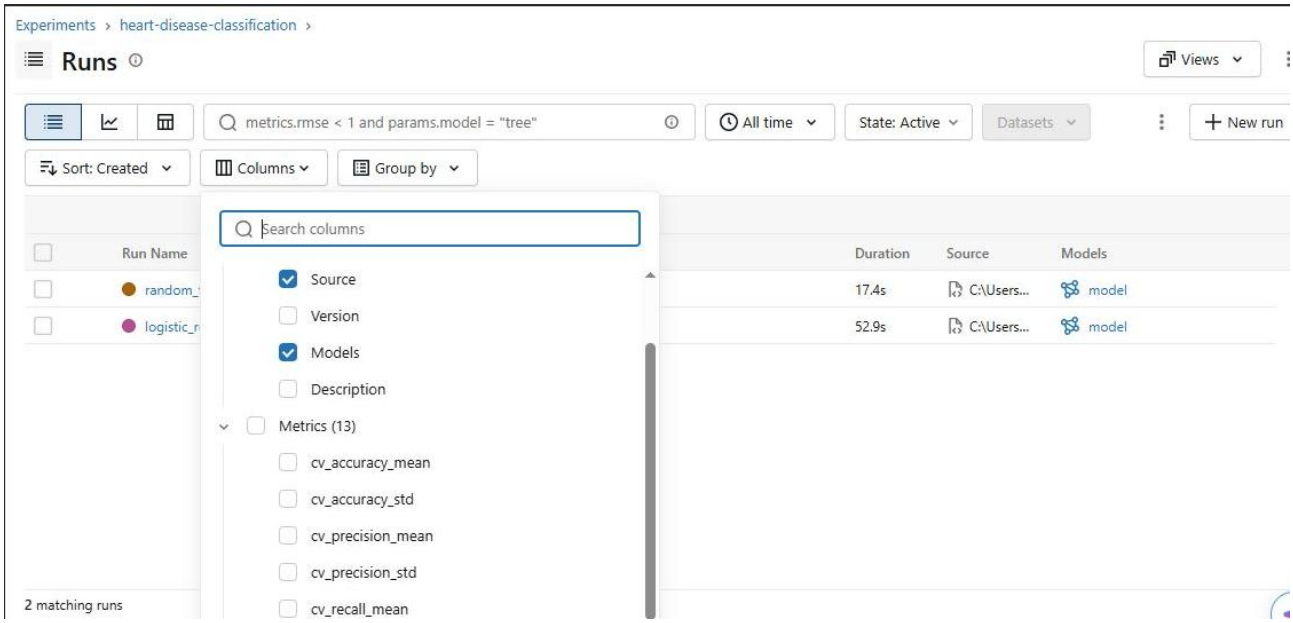


Figure 8. MLflow experiment page with the two runs.

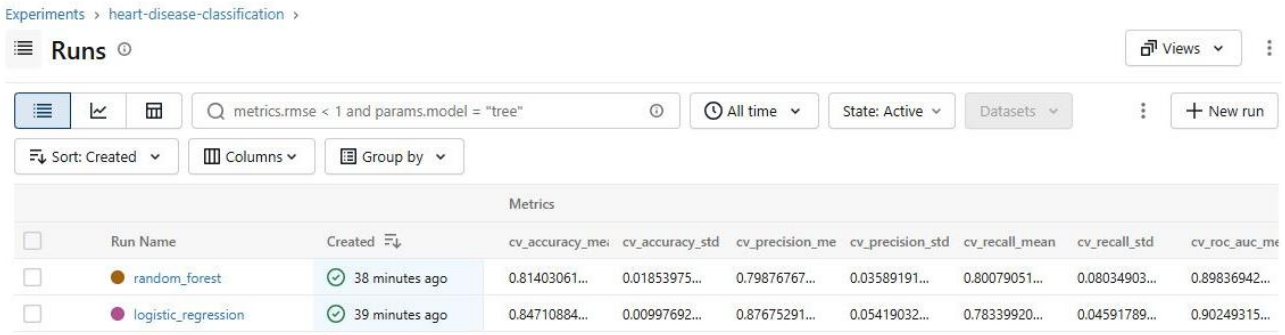


Figure 9. Side-by-side comparison of the Logistic Regression and Random Forest runs.

heart-disease-classification › Runs ›

logistic_regression

Overview | Model metrics | System metrics | Traces | Artifacts

cv_accuracy_mean	0.8471088435374149	model
cv_accuracy_std	0.009976921631744011	model
cv_precision_mean	0.8767529146045769	model
cv_precision_std	0.054190324098361047	model
cv_recall_mean	0.783399209486166	model
cv_recall_std	0.045917899546111016	model
cv_roc_auc_mean	0.9024931590148981	model
cv_roc_auc_std	0.01443995119614911	model
test_accuracy	0.8852459016393442	model
test_precision	0.8387096774193549	model
test_recall	0.9285714285714286	model
test_f1	0.8813559322033898	model
test_roc_auc	0.9664502164502166	model

Figure 10. Run detail: parameters and metrics.

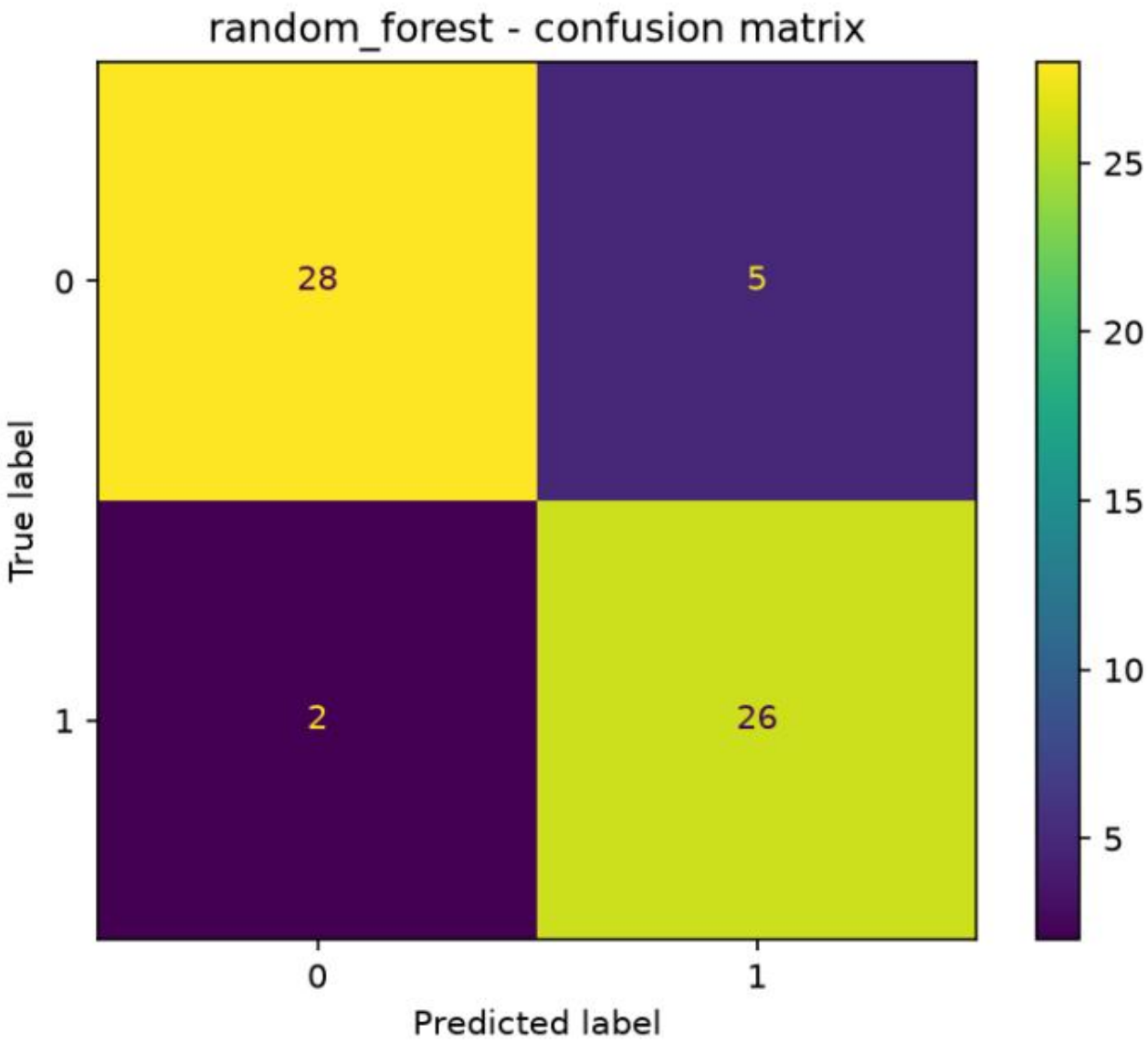


Figure 11. Run artifacts: model, confusion matrix and ROC curve.

6. Model packaging and reproducibility

The final artifact is the complete pipeline (preprocessing and classifier) saved with joblib at `models/model.joblib`, and also as an MLflow model. Because the preprocessing travels with the model, training and serving apply identical transformations.

`src/predict.py` loads the artifact and returns the predicted class, a confidence score (the larger of the two class probabilities) and the probability of disease. The API reuses this module. For a sample patient (the first row of the dataset, true label 0) the model returned prediction 0 with probability of disease 0.1655.

7. Automated testing and CI/CD

Nine pytest tests live in `tests/`:

- **Preprocessing** (4 tests): output shape, handling of missing values, scaled numeric features, unseen categories.
- **Model** (5 tests): output keys, binary prediction, confidence range, DataFrame input, and an error when a feature is missing.

The GitHub Actions workflow `.github/workflows/ci.yml` runs on every push and pull request to main: install dependencies, lint with flake8, run pytest (JUnit report), retrain the model, and upload the test results, model and training plots as artifacts. A failing step fails the run, with logs showing the error.

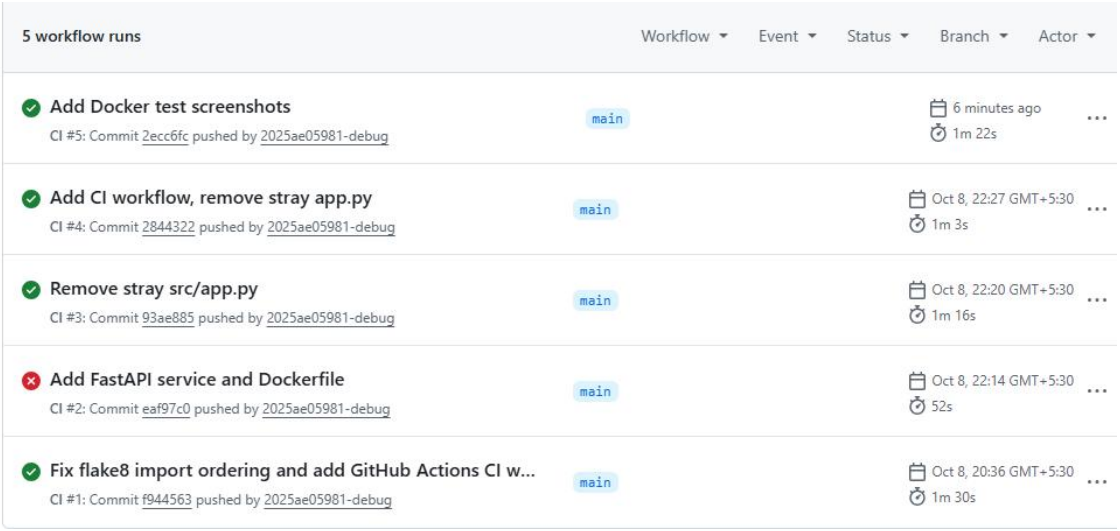


Figure 12. CI run history. Run #2 failed on lint errors and run #3 fixed them.

Pipeline failure behaviour. Run #2 failed because a stray file (`src/app.py`) was committed with lint errors (E501, W293, W292). The pipeline stopped at the lint step. Removing the file in the next commit turned the run green, which demonstrates that the pipeline fails on code errors and reports them clearly.

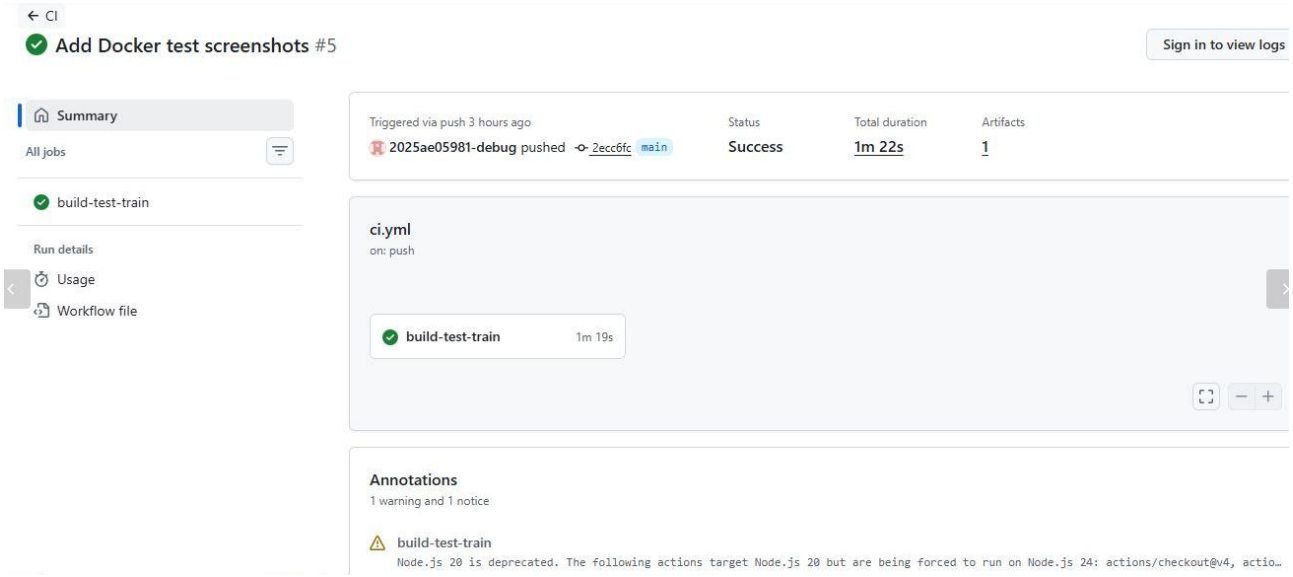


Figure 13. Steps of a passing CI run (install, lint, test, train, upload).

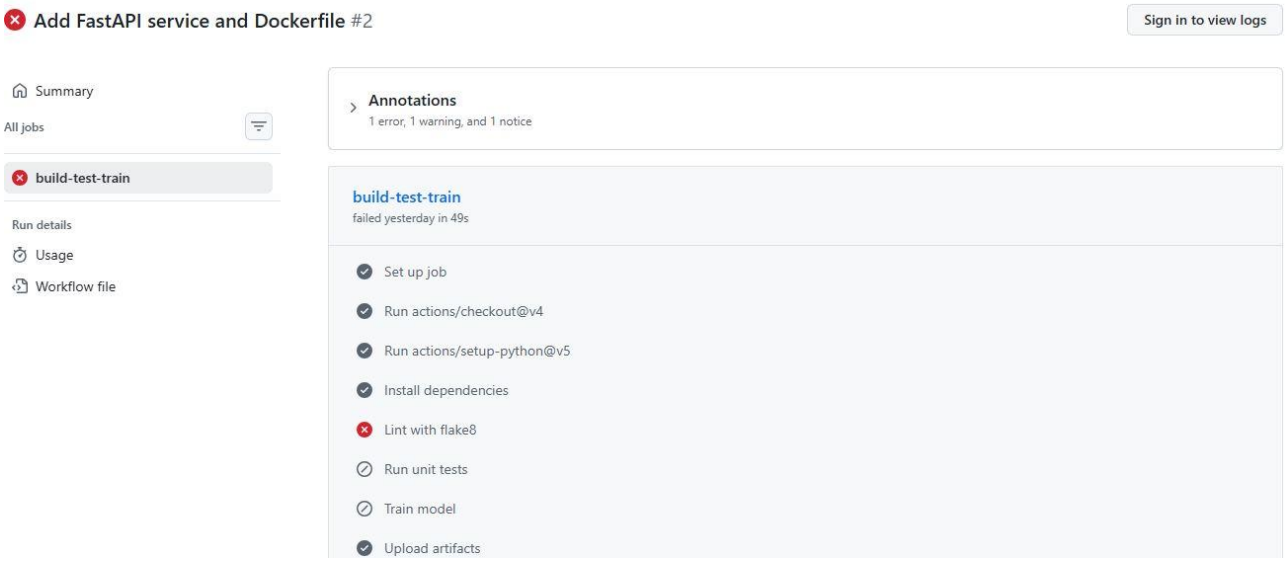


Figure 14. Log of the failed lint step in run #2.

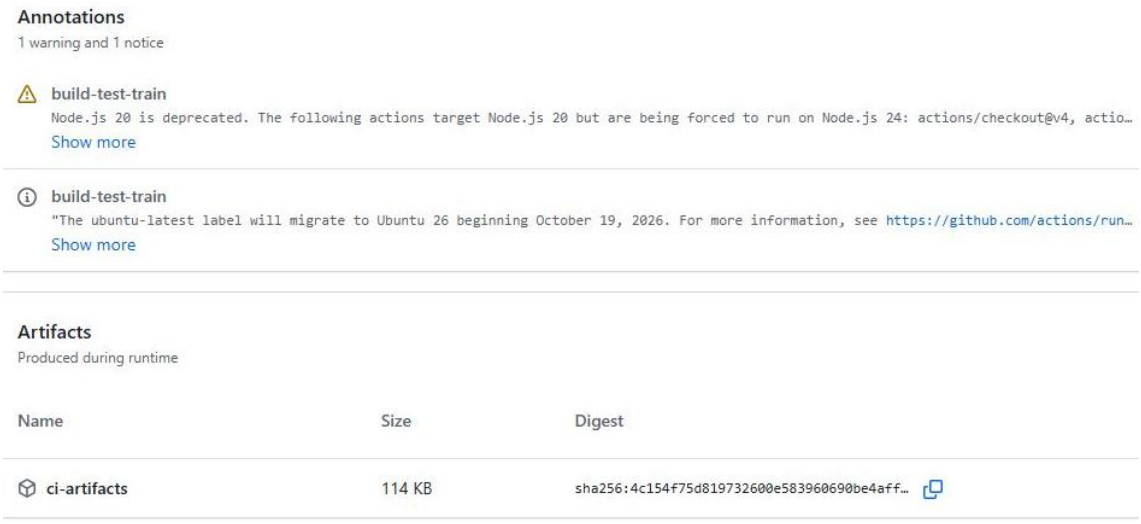


Figure 15. Artifacts uploaded by the CI run.

8. Model containerisation

The model is served by a FastAPI application (`api/main.py`) with these endpoints:

Endpoint	Method	Purpose
/predict	POST	JSON patient record in; prediction, confidence and probability out
/health	GET	Liveness and readiness check
/metrics	GET	Prometheus metrics
/docs	GET	Interactive Swagger UI

Input is validated with Pydantic, with realistic ranges per field. Invalid requests, such as an age of 500, are rejected with HTTP 422 and a message naming each problem. Every request is logged with its method, path, status and latency, and each prediction is logged with its confidence.

The Dockerfile uses `python:3.13-slim`, installs only the serving requirements, copies `src/`, `api/` and `models/`, and starts uvicorn on port 8000. The resulting image is about 668 MB on disk.

```
docker build -t heart-api:latest .
docker run -d -p 8000:8000 --name heart-api heart-api:latest
```

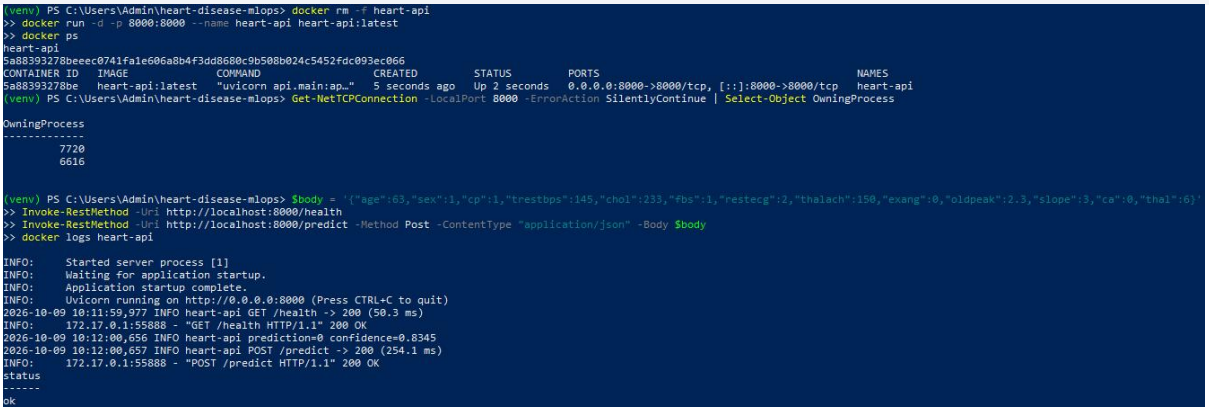


Figure 16. Container running (docker ps), the /predict response, and the container logs.

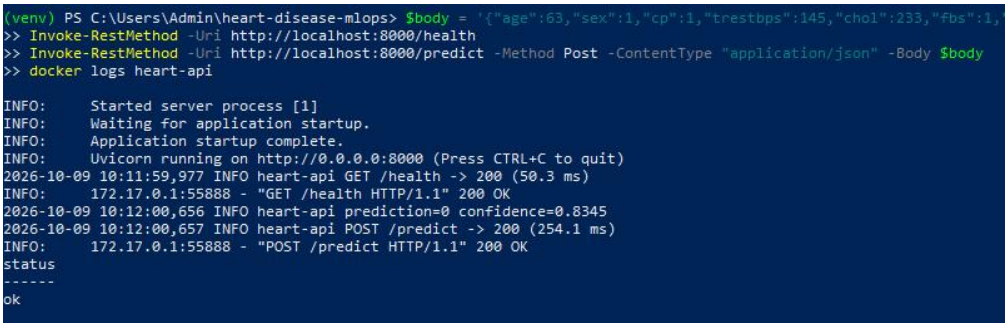


Figure 17. Container logs showing GET /health and POST /predict returning 200.

9. Production deployment on Kubernetes

The image was deployed to the local Kubernetes cluster that ships with Docker Desktop (kubeadm, single node, v1.36.1) using two manifests in `k8s/`.

- **deployment.yaml:** 2 replicas, readiness and liveness probes on `/health`, CPU and memory requests and limits, and `imagePullPolicy: Never` so the locally built image is used.
- **service.yaml:** a LoadBalancer service on port 80 forwarding to container port 8000. On Docker Desktop its external address is `localhost`.

```
kubectl apply -f k8s\deployment.yaml
kubectl apply -f k8s\service.yaml
kubectl get pods
kubectl get service heart-api-service
```



Figure 18. Cluster node in Ready state.

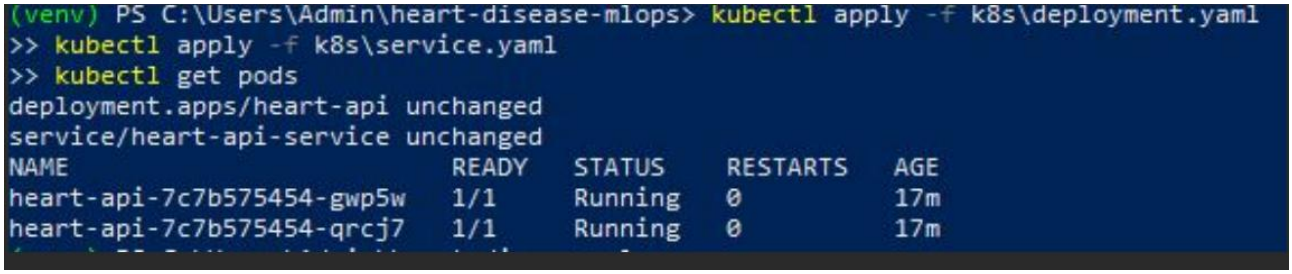


Figure 19. Two heart-api pods running (1/1, no restarts).

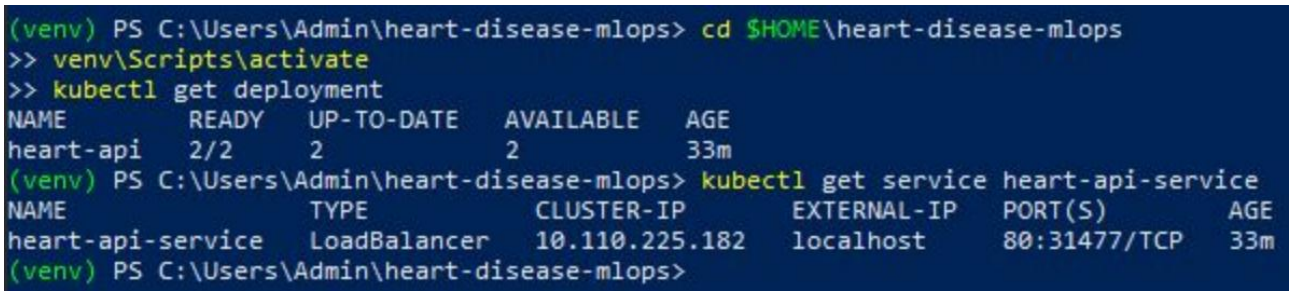


Figure 20. Deployment (2/2) and LoadBalancer service.

```
(venv) PS C:\Users\Admin\heart-disease-mlops> cd $HOME\heart-disease-mlops
>> venv\Scripts\activate
>> kubectl get deployment
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
heart-api   2/2     2            2           33m
(venv) PS C:\Users\Admin\heart-disease-mlops> kubectl get service heart-api-service
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
heart-api-service   LoadBalancer   10.110.225.182   localhost     80:31477/TCP   33m
(venv) PS C:\Users\Admin\heart-disease-mlops> $body = '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,
>> Invoke-RestMethod -Uri http://localhost/health
>> Invoke-RestMethod -Uri http://localhost/predict -Method Post -ContentType "application/json" -Body $body

status
------
ok



(venv) PS C:\Users\Admin\heart-disease-mlops> cd $HOME\heart-disease-mlops
>> venv\Scripts\activate
>> $body = '{"age":63,"sex":1,"cp":1,"trestbps":145,"chol":233,"fbs":1,"restecg":2,"thalach":150,"exang":0,"oldpeak":2.3,"slope":3,"ca":0,"thal":6}'
>> Invoke-RestMethod -Uri http://localhost/predict -Method Post -ContentType "application/json" -Body $body

prediction confidence probability_disease
---------- ---------- ------------------
         0     0.8345             0.1655
```

Figure 21. Endpoint verification through the LoadBalancer: /health and /predict.

```
(venv) PS C:\Users\Admin\heart-disease-mlops> kubectl logs -l app=heart-api --tail=200 | Select-String "predict"

2026-10-09 15:45:42,175 INFO heart-api prediction=0 confidence=0.8345
2026-10-09 15:45:42,176 INFO heart-api POST /predict -> 200 (245.2 ms)
INFO:     192.168.65.3:38714 - "POST /predict HTTP/1.1" 200 OK
2026-10-09 15:47:33,627 INFO heart-api prediction=0 confidence=0.8345
2026-10-09 15:47:33,628 INFO heart-api POST /predict -> 200 (16.7 ms)
INFO:     192.168.65.3:57582 - "POST /predict HTTP/1.1" 200 OK
2026-10-09 15:47:33,807 INFO heart-api prediction=0 confidence=0.8345
2026-10-09 15:47:33,809 INFO heart-api POST /predict -> 200 (153.9 ms)
INFO:     192.168.65.3:57582 - "POST /predict HTTP/1.1" 200 OK
2026-10-09 15:47:33,848 INFO heart-api prediction=0 confidence=0.8345
2026-10-09 15:47:33,850 INFO heart-api POST /predict -> 200 (34.7 ms)
INFO:     192.168.65.3:57582 - "POST /predict HTTP/1.1" 200 OK
2026-10-09 15:47:33,923 INFO heart-api prediction=0 confidence=0.8345
2026-10-09 15:47:33,924 INFO heart-api POST /predict -> 200 (67.0 ms)
INFO:     192.168.65.3:57582 - "POST /predict HTTP/1.1" 200 OK
2026-10-09 15:47:33,944 INFO heart-api prediction=0 confidence=0.8345
2026-10-09 15:47:33,945 INFO heart-api POST /predict -> 200 (17.1 ms)
INFO:     192.168.65.3:57582 - "POST /predict HTTP/1.1" 200 OK
2026-10-09 15:47:34,052 INFO heart-api prediction=0 confidence=0.8345
2026-10-09 15:47:34,053 INFO heart-api POST /predict -> 200 (105.5 ms)
INFO:     192.168.65.3:57582 - "POST /predict HTTP/1.1" 200 OK
```

Figure 22. Pod logs showing POST /predict requests served with status 200.

10. Monitoring and logging

The API exposes three Prometheus metrics: `api_requests_total` (by method, path and status), `api_request_latency_seconds` (histogram by path) and `api_predictions_total` (by predicted class). Request and prediction logging is written to standard output and can be read with `kubectl logs`.

Prometheus and Grafana run in the same cluster (`monitoring/`). Because there are two replicas, a headless service lets Prometheus scrape each pod separately. Grafana is pre-configured with Prometheus as its data source, and the dashboard is exported to `monitoring/grafana-dashboard.json`.

Dashboard panels

Panel	PromQL
Request rate	sum(rate(api_requests_total[1m])) by (path)
Predictions by class	sum(api_predictions_total) by (prediction)
Average latency	rate of latency sum / rate of latency count for /predict
p95 latency	histogram_quantile(0.95, ... api_request_latency_seconds_bucket ...)
Error responses	sum(rate(api_requests_total{status=~"4..|5.."}[1m]))

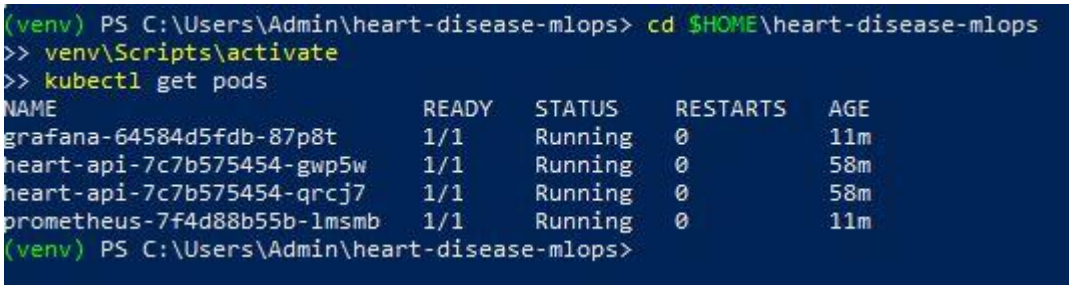


Figure 23. Application and monitoring pods running.

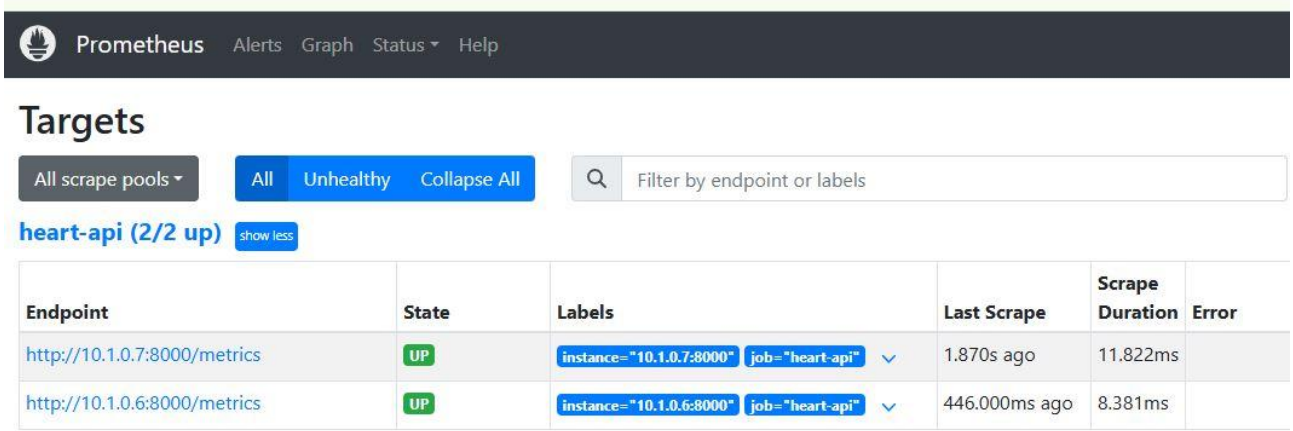


Figure 24. Prometheus targets: both API pods are UP.

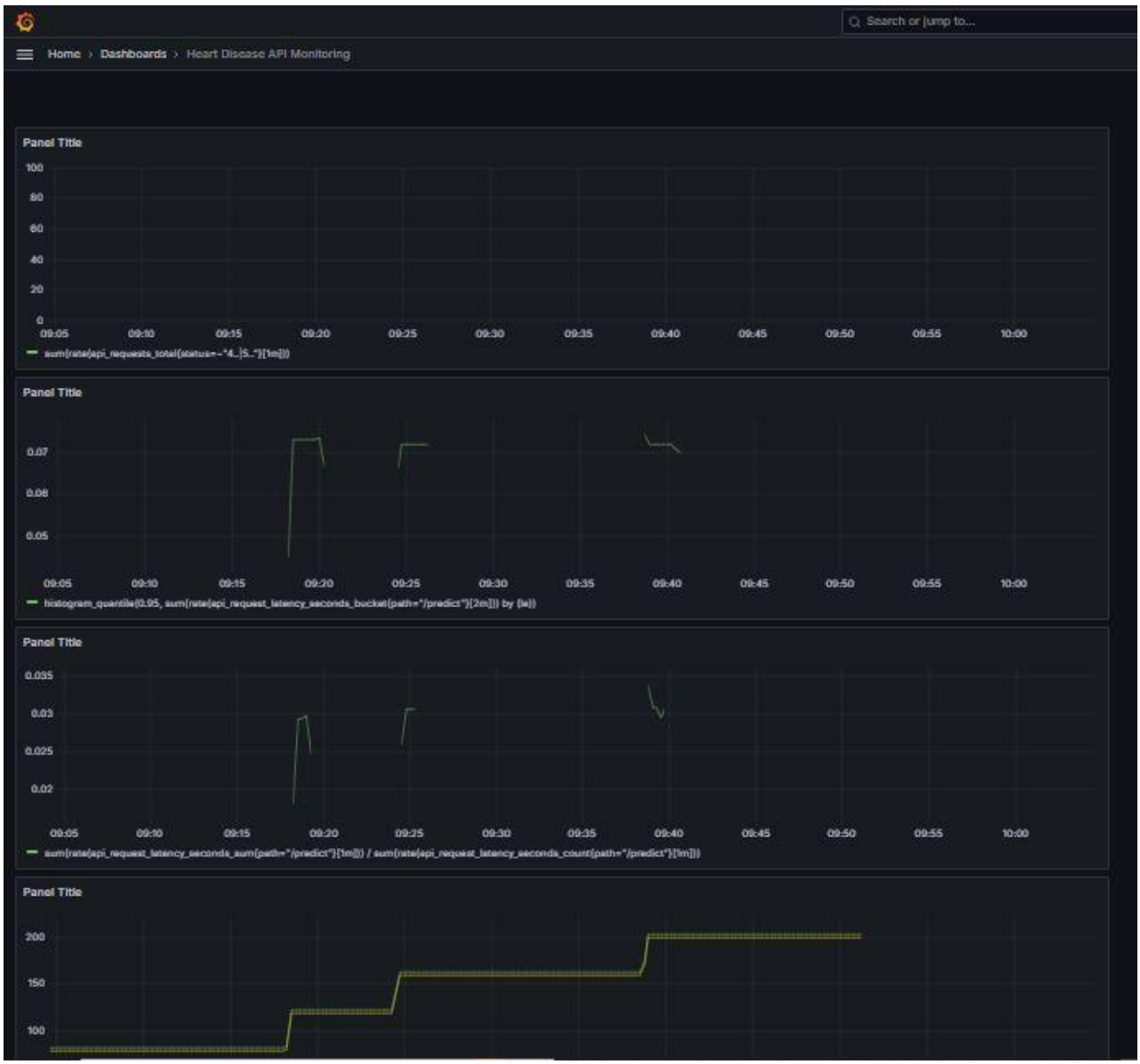
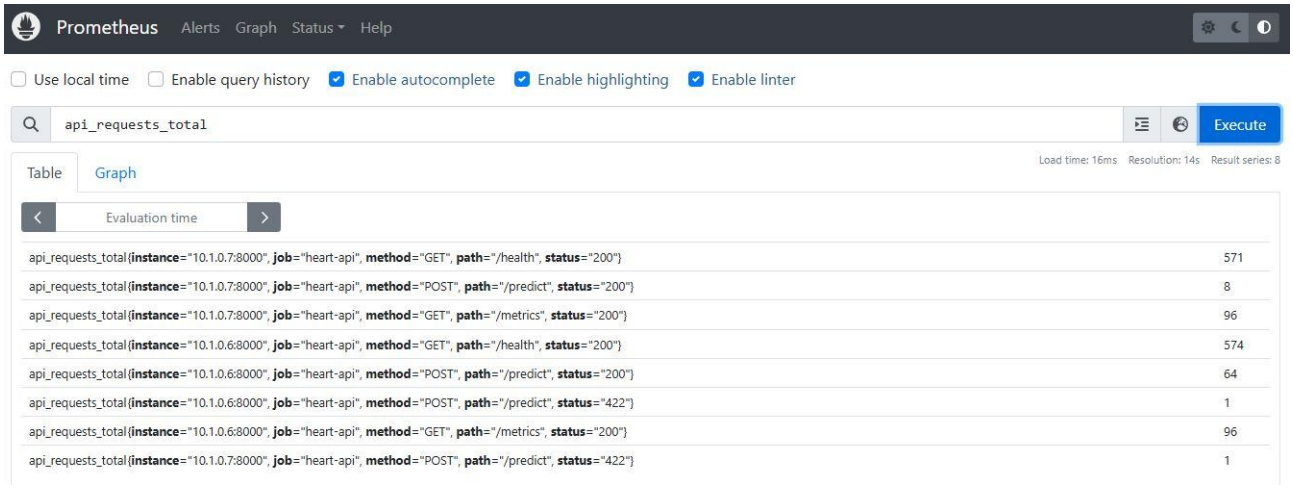


Figure 25. Grafana dashboard with live traffic.



api_requests_total{**instance**="10.1.0.7:8000", **job**="heart-api", **method**="GET", **path**="/health", **status**="200"}	571
api_requests_total{**instance**="10.1.0.7:8000", **job**="heart-api", **method**="POST", **path**="/predict", **status**="200"}	8
api_requests_total{**instance**="10.1.0.7:8000", **job**="heart-api", **method**="GET", **path**="/metrics", **status**="200"}	96
api_requests_total{**instance**="10.1.0.6:8000", **job**="heart-api", **method**="GET", **path**="/health", **status**="200"}	574
api_requests_total{**instance**="10.1.0.6:8000", **job**="heart-api", **method**="POST", **path**="/predict", **status**="200"}	64
api_requests_total{**instance**="10.1.0.6:8000", **job**="heart-api", **method**="POST", **path**="/predict", **status**="422"}	1
api_requests_total{**instance**="10.1.0.6:8000", **job**="heart-api", **method**="GET", **path**="/metrics", **status**="200"}	96
api_requests_total{**instance**="10.1.0.7:8000", **job**="heart-api", **method**="POST", **path**="/predict", **status**="422"}	1

Figure 26. Prometheus query of api_requests_total.

11. Problems encountered and lessons learned

Problem	Cause	Resolution
MLflow refused to start the file store	MLflow 3.16 puts the mlruns file backend in maintenance mode	Switched the tracking URI to SQLite
MLflow refused to log the model (untrusted type numpy.dtype)	New skops security check on scikit-learn models	Logged with the cloudpickle serialization format
flake8 reported thousands of errors	It was scanning the venv folder and Jupyter checkpoints	Configured excludes and linted only project folders
E402 in train.py	matplotlib.use() was placed between imports	Moved it after the import block
CI run #2 failed	Stray src/app.py with lint errors was committed	Removed the file; run #3 passed
API answered, but the container did not exist	A local uvicorn process was still using port 8000	Stopped the local server, then ran and tested the container
docker commands could not connect	Docker Desktop engine was not running	Started Docker Desktop first

Key lessons: keep preprocessing inside the saved pipeline, pin dependencies, verify that the container (not a local process) answers requests, and treat a failed pipeline run as useful evidence that the checks work.

12. Conclusion

The project delivers the full path from raw data to a monitored production-style service: a reproducible preprocessing and modelling pipeline with tracked experiments (Logistic Regression, CV ROC-AUC 0.902), automated tests and CI, a Docker image, a two-replica Kubernetes deployment behind a LoadBalancer, and Prometheus and Grafana monitoring. Possible next steps are model versioning with the MLflow registry, drift monitoring on the input features, a public cloud deployment, and autoscaling.

Links

Repository: https://github.com/2025ae05981-debug/heart-disease-mlops

Demo video: [add your video link here]

Access: after applying the manifests, the API is at http://localhost (or http://localhost:8080 with `kubectl port-forward service/heart-api-service 8080:80`).